Report for the Artificial Intelligence (CES571)

Huajian Liu
Arizona State University
Tempe, US
huajianl@asu.edu

Abstract—This report is intended to summarize the three projects from the Artificial Intelligence course.

I. Introduction

This course primarily focused on techniques related to Artificial Intelligence, including supervised learning, unsupervised learning, and deep learning, with an in-depth exploration of the deep learning domain. Deep learning is a subfield of machine learning that employs neural networks with multiple layers to model complex patterns in data. By mimicking the structure and function of the human brain, deep learning algorithms automatically extract hierarchical features from raw data, enabling applications such as image recognition, natural language processing, and speech recognition. The training process involves feeding large datasets into the model, which uses a technique called backpropagation to adjust weights and minimize prediction errors. While deep learning excels in handling large-scale data and achieving high accuracy in various tasks, it also faces challenges such as requiring significant computational resources, data, and a lack of interpretability in its decision-making processes. Overall, deep learning is a powerful tool that has significantly advanced the capabilities of artificial intelligence.We utilize a project to explain every detail of deep learning techniques.

II. Project : Neural Network for Collision Prediction

A. Project Description

In this project, our goal is to help a small robot navigate a simulated environment without colliding with any obstacles. To achieve this, we need to utilize backpropagation to train the neural network so that it can predict whether the robot will collide with any walls or objects in the next time step. All the theories and best practices introduced in the course will apply to this assignment[1]. Our task is to ensure that this small robot can safely explore its environment.

B. Part 1: collecting data

The first task is to collect data that can be used to train the model. For each action, collect a sample that sequentially includes 5 distance sensor readings, the action, and whether a collision occurred (0: no collision, 1: collision).This data was saved as a .csv file containing 7 unlabeled columns. We use the following code to implement this functionality.

```
def collect_training_data(total_actions):
  #set-up environment
  global sensor_readings
  sim_env = sim.SimulationEnvironment()

  #robot control
  action_repeat = 100
  steering_behavior = Wander(action_repeat)

  num_params = 7
  network_params =
np.zeros((total_actions,num_params),dtype="float32")

  for action_i in range(total_actions):
    progress = 100*float(action_i)/total_actions
    print(f'Collecting Training Data {progress}%   ', end="\r",
flush=True)
    #steering_force is used for robot control only
    action, steering_force = steering_behavior.get_action(action_i,
sim_env.robot.body.angle)
    for action_timestep in range(action_repeat):
      if action_timestep == 0:
        _, collision, sensor_readings = sim_env.step(steering_force)
      else:
        _, collision, _ = sim_env.step(steering_force)
      if collision:
        steering_behavior.reset_action()
        #if current action is very new.
        if action_timestep < action_repeat * .3: #in case prior action
caused collision
          network_params[action_i][-1] = collision #share collision
result with prior action
        break
    network_params[action_i][0:-2] = sensor_readings
    network_params[action_i][-2] = action

  count_of_ones = np.sum(network_params[:, -1] == 1)
  np.savetxt("submission.csv", network_params, delimiter=',')
```

The function `collect_training_data(total_actions)` is defined to collect training data for a robot's actions in a simulated environment.

1. **Environment Setup**: A global variable `sensor_readings` is declared, and an instance of `SimulationEnvironment` is created to simulate the robot's environment.
2. **Robot Control Initialization**:`action_repeat` is set to 100, meaning each action will be repeated 100 times. `Wander(action_repeat)` initializes a steering behavior that defines how the robot will move.
3. **Data Structure Initialization**:`num_params` is set to 7, indicating the number of parameters to be collected for each sample. `network_params` is

an array initialized to zeros, with dimensions `(total_actions, num_params)`, that will store the training data.

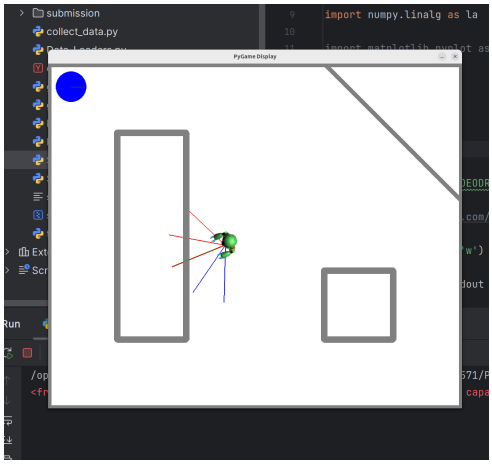
4. **Data Collection Loop**:A loop iterates over the range of `total_actions`, calculating and printing the progress of data collection. For each action, the steering force is obtained, and another loop runs for the specified number of repetitions (`action_repeat`).

5. **Simulation Steps**:On the first timestep, the simulation step returns sensor readings and checks for collisions. For subsequent timesteps, it only checks for collisions. If a collision occurs, the steering behavior is reset, and if the collision happens early in the action (first 30%), the collision status is recorded for the previous action.

6. **Data Update**:After the inner loop, the collected sensor readings, the action, and the collision status are stored in `network_params`.

7. **Data Saving**:A count of how many times a collision occurred is printed. Finally, the collected data is saved to a CSV file named `"submission.csv"` using `numpy.savetxt`.

Image 1 cellecting data



C. *Part2: Data Operations*

We have already collected the training data, and now we need to develop a PyTorch DataLoader to make it easier to use the data. We will prune the collected data to balance the data distribution[2]. We will create a DataLoader and a test DataLoader. We will use the `training_data.csv` collected in the previous section for testing.

`Nav_Dataset` class:

1. Loads a CSV file containing data with `np.genfromtxt` and splits it into "collision" (where the last column is 1) and "no collision" (where the last column is 0) data.
2. Balances the dataset by randomly selecting a number of "no collision" samples equal to the number of "collision" samples.
3. Concatenates and shuffles the data.
4. Normalizes the data using a `MinMaxScaler` and saves the scaler to a file for later use in inference.
5. Implements the `__len__()` and `__getitem__()` methods to make the dataset compatible with PyTorch data loaders. The `__getitem__()` method returns a dictionary with 'input' and 'label' keys, where both are converted to `float32`.

`Data_Loaders` class:

1. Initializes a `Nav_Dataset` object and splits it into training (80%) and testing (20%) datasets using `data.random_split`.
2. Creates `train_loader` and `test_loader` using PyTorch's `DataLoader` to handle batching during training and evaluation.

```
class Nav_Dataset(dataset.Dataset):
  def __init__(self):
    self.data = np.genfromtxt('saved/training_data.csv',
delimiter=',')
    # get the collision and no_collision data
    self.collision_data = self.data[np.where(self.data[:, -1] == 1)]
    self.no_collision_data = self.data[np.where(self.data[:, -1] ==
0)]
    # get random indices
    self.ratio = 1
    random_indices =
np.random.choice(self.no_collision_data.shape[0],
                     size=self.ratio
*self.collision_data.shape[0],
                     replace=False)
    self.no_collision_data =
self.no_collision_data[random_indices]
    # concatenate the collision data and no collision data
    self.data = np.concatenate((self.collision_data,
self.no_collision_data))
    # shuffle the data
    np.random.shuffle(self.data)
    # normalize data and save scaler for inference
    self.scaler = MinMaxScaler()
    self.normalized_data = self.scaler.fit_transform(self.data)
#fits and transforms
    pickle.dump(self.scaler, open("saved/scaler.pkl", "wb")) #save
to normalize at inference
  def __len__(self):
    return len(self.normalized_data)
  def __getitem__(self, idx):
    if not isinstance(idx, int):
      idx = idx.item()
    input_features = self.normalized_data[idx][:-1]
    label = self.normalized_data[idx][-1]
    return {'input': input_features.astype(float32), 'label':
label.astype(float32)}


class Data_Loaders():
  def __init__(self, batch_size):
    self.batch_size = batch_size
    self.nav_dataset = Nav_Dataset()
    dataset_size = len(self.nav_dataset)
    # 80% for training
    train_samples_size = int(0.8 * dataset_size)
    # 20% for testing
    test_samples_size = dataset_size - train_samples_size
    # Randomly split the dataset into training and testing
datasets
    train_samples_data, test_samples_data =
data.random_split(self.nav_dataset, [train_samples_size,
test_samples_size])
    self.train_loader = data.DataLoader(train_samples_data,
batch_size)
    self.test_loader = data.DataLoader(test_samples_data)
```

D. *Part3: make the network model*

In this part, We will use PyTorch's `torch.nn` class to design my own custom neural network[3]. I need to initialize a custom architecture, define the forward pass in the network, and build a method to evaluate how well the given model fits the data.

```
class Action_Conditioned_FF(nn.Module):
  def __init__(self):
    # custom architecture
    super(Action_Conditioned_FF, self).__init__()

    # the schema of the input layer
    self.input_layer = nn.Linear(6, 300)
    # the schema of the second layer
    self.second_layer = nn.Linear(300, 400)
    # the schema of the output layer
    self.outputLayer = nn.Linear(400, 1)
    self.ReLU = nn.ReLU()

    self.Sigmoid = nn.Sigmoid()

    # init the method of regulation
    self.dropout_prob = 0.5
    self.dropout = nn.Dropout(p=self.dropout_prob)

    pass

  def forward(self, input):
    # init the network forward process
    first = self.ReLU(self.input_layer(input))
    self.dropout(first)
    second = self.ReLU(self.second_layer(first))
    self.dropout(second)

    output = torch.flatten(self.Sigmoid(self.outputLayer(second)))
    return output

  def evaluate(self, model, test_loader, loss_function):
    # output and the desired output.
    loss = 0.0
    len = 0
    for idx, sample in enumerate(test_loader):
      input_features, label = sample['input'], sample['label']

      pred_result = model(input_features)
      loss += loss_function(pred_result, label).item()
      len += 1
    return loss / len
```

This code defines a custom feedforward neural network (`Action_Conditioned_FF`) using PyTorch.

1. **Initialization (`__init__`)**:
   - **Layers**: Defines three fully connected layers:
     - `input_layer` (6 input features → 300 units)
     - `second_layer` (300 units → 400 units)
     - `outputLayer` (400 units → 1 output)
   - **Activations**: Uses ReLU for hidden layers and Sigmoid for the output.
   - **Regularization**: Applies dropout with a 50% probability after each hidden layer.
2. **Forward Pass (`forward`)**:
   - Input passes through the `input_layer`, ReLU activation, dropout, `second_layer`, another ReLU, dropout, and finally through the `outputLayer` with Sigmoid activation.
   - The output is flattened and returned.
3. **Evaluation (`evaluate`)**:
   - Loops over the test dataset, computes the loss for each batch, and returns the average loss.

E. *Part4: train model*

We use the code bellow to train the model.

```
def train_model_mini_batch(no_epochs):
  batch_size = 800
  data_loaders = Data_Loaders(batch_size)
  model = Action_Conditioned_FF()

  loss_function = nn.MSELoss()
  trainLosses = []
  validateLosses = []

  optimizer = torch.optim.Adam(model.parameters(), lr=0.0001)

  for epoch_i in range(no_epochs):
    model.train()
    optimizer.zero_grad()
    for idx, sample in enumerate(data_loaders.train_loader):  #
sample['input'] and sample['label']
      input_features, label = sample['input'], sample['label']
      outputs = model(input_features)
      loss = loss_function(outputs, label)
      trainLosses.append(loss.item())
      loss.backward()
      optimizer.step()
      # if idx % 10 == 9:
      print('[%d, %5d] loss: %.3f' % (epoch_i + 1, idx + 1, loss))

      model.eval()
      val_loss = model.evaluate(model, data_loaders.test_loader,
loss_function)
      validateLosses.append(val_loss)

  torch.save(model.state_dict(), 'saved/saved_model.pkl',
_use_new_zipfile_serialization=False)

  return trainLosses, validateLosses
```

The function `train_model_mini_batch(no_epochs)` trains a neural network model using mini-batch gradient descent over a specified number of epochs. It initializes the model, loss function (Mean Squared Error), and Adam optimizer, then iterates through the training data in batches. For each batch, it performs a forward pass through the model, computes the loss, performs backpropagation, and updates the model's weights[4]. The training loss is recorded for each batch, while the model is also evaluated on a validation set after each epoch to monitor its performance on unseen data. The function then saves the trained model's parameters todiskand returns the lists of training and validation losses.This process allows the model to learn from the data, refine its parameters over time, and track both training and validation performance.

F. *Results Analysis*

The image below shows the training and validation loss during the model's training. The x-axis represents the number of training and test samples, while the y-axis represents the loss.

Image2 training and validation loss

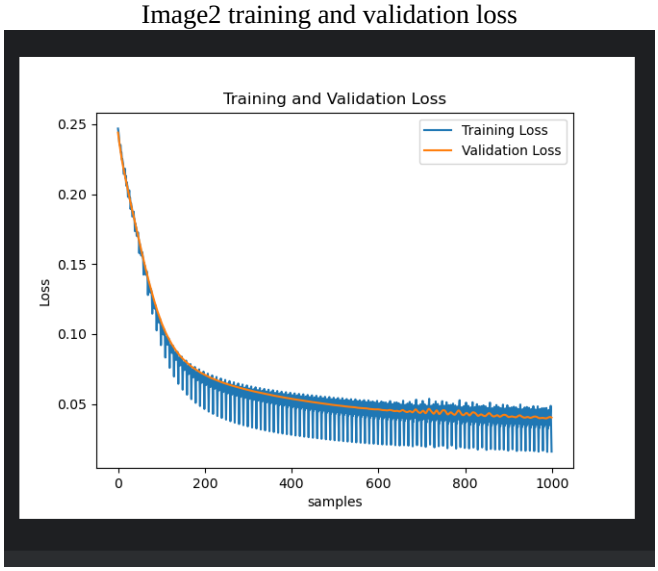
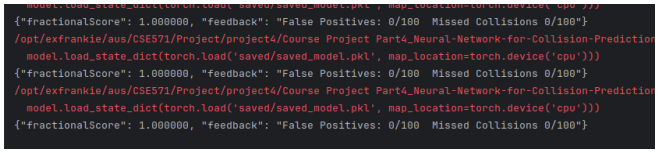


Image3 grader result



We use the a function `evaluate_accuracy(total_actions)` evaluates the accuracy of a trained neural network model by simulating actions in a robot environment, where the goal is to predict collisions based on the robot's actions. It first loads the trained model (`Action_Conditioned_FF`) and normalization parameters (scaler) from saved files. The function then uses a `Wander` steering behavior (or optionally a `Seek` behavior) to generate actions for the robot, feeding these actions into the model to predict collision outcomes. It compares the model's predictions with actual collisions during a series of simulated steps and tracks accurate predictions, false positives, and missed collisions. Based on these counts, it calculates a score that penalizes missed collisions and excessive false positives. The final score and feedback, including statistics on false positives and missed collisions, are printed in JSON format. The `evaluate_accuracy` function is called repeatedly over multiple experiments (20 times in the `if __name__ == '__main__'` block) to assess the model's performance across several runs.

III. References

[1] Hand D. J. ,Yu. K. (2001),“Idiot's Bayes — not so stupid after all”,International Statistical Review. 69 (3): 385–399.

[2] Education Ecosystem (LEDU),“Understanding K-means Clustering in Machine Learning”, online at: https://towardsdatascience.com/understanding-k-means-clustering-in-machine-learning-6a6e67336aa1

[3] Ashley,“An Overview on Convolutional Neural Networks”,online at: https://medium.com/swlh/an-overview-on-convolutional-neural-networks-ea48e76fb186

[4] M. Jordan,J. Kleinberg,B. Scho¨lkopf,“Pattern Recognition and Machine Learning”, Springer--2006